

Attention-Augmented Scale Hyperprior for Learned Image Compression

Emre Yilmaz
CS566 Deep Learning, Spring 2026

Abstract

I studied whether lightweight channel attention can improve learned image compression when added to a standard scale-hyperprior architecture. Starting from the official CompressAI implementation of Ballé et al.’s scale hyperprior, I implemented two Squeeze-and-Excitation (SE) variants: encoder-only attention and encoder+decoder attention. I kept the hyperprior unchanged so that I could test the attention modules as a focused architectural change. Compared with the original proposal, the final training pipeline uses explicit CLIC train/validation image folders rather than DIV2K, and it adds a fine-tuned vanilla baseline to make the comparison fair.

Training was performed on 1,633 CLIC training images with 102 validation images using random 256×256 crops, and evaluation was carried out on the 24-image Kodak benchmark. I report PSNR, MS-SSIM, estimated bitrate, and true coded bitrate obtained from actual `compress()`/`decompress()` calls. This turned out to be important during project cleanup: earlier plots based only on estimated bitrate made the SE variants look better than they really were, while the corrected coded-bitrate evaluation shows that the curves are nearly identical.

The final results do not show a meaningful rate-distortion gain from the tested SE modification. Relative to the fine-tuned vanilla baseline, encoder-only SE yields a PSNR BD-rate of +0.08% and encoder+decoder SE yields +0.17%; the MS-SSIM differences are also very small. Across the four operating points, the project variants stay within 0.009 dB and 5.8×10^{-4} bpp of each other. Stronger entropy models such as MBT 2018 and Cheng 2020 provide much larger gains. So, within the scope of this course project, the main conclusion is that small transform-side SE attention is easy to integrate, but it does not noticeably improve compression performance in this setting.

1 Introduction

Learned image compression replaces fixed transforms and hand-designed probability models with neural networks trained directly for rate-distortion optimization. This is useful because the encoder, decoder, and entropy model can be learned together from data instead of being designed separately. In recent years, learned compression models have become strong enough that even small architectural changes can be tested in a meaningful way, which makes them a good fit for a graduate course project.

The starting point of this project was the scale hyperprior model of Ballé et al. [2], implemented through CompressAI [6]. The original proposal asked a simple question: if I keep the hyperprior unchanged and only add lightweight attention inside the transform network, do I get better rate-distortion behavior? This is a modest goal, but it is still useful because it isolates the effect of the attention block from bigger changes to the entropy model.

The final repository stays close to that plan at the model level. I implemented two attention-augmented variants: one with SE attention in the encoder and one with SE attention in both encoder and decoder. At the same time, the experimental pipeline evolved in two important ways. First, the final training runs use CLIC train/validation splits instead of the initially planned DIV2K training set. Second, I added a fine-tuned vanilla control baseline because comparing SE variants only against the frozen official pretrained model would not be a fair test of the modification.

This report focuses on two questions. First, did the implementation actually match the proposed modification?

Table 1: Datasets found in the repository and their final role in the project.

Dataset	Role	Count	Resolution	Final use
CLIC train	training	1,633	variable	yes
CLIC validation	validation	102	variable	yes
Kodak PhotoCD	test benchmark	24	mostly 768×512	yes
DIV2K train	alternate train set	800	high-resolution	no
DIV2K validation	alternate val set	100	high-resolution	no

Second, after using the corrected evaluation pipeline and fair baselines, do the SE blocks improve image compression on Kodak? The short answer is that the implementation matches the proposal reasonably well, but the experimental evidence for improvement is weak.

Project repository: [emreYilmaz28845/LearnedCompression.git](https://github.com/emreYilmaz28845/LearnedCompression.git).

2 Related Work

Classical codecs such as JPEG and later transform-based codecs rely on carefully designed transforms, quantization rules, and entropy coders. Their main strength is robustness and efficiency, but the transform and prior model are fixed in advance. Learned compression instead treats compression as a rate-distortion learning problem, so the transform and latent prior can adapt to the training data.

An early end-to-end formulation was introduced by Ballé et al. [1], who showed that nonlinear transforms with a differentiable rate term can be optimized directly. The scale hyperprior model [2] improved this idea by introducing a second latent variable z that predicts spatially varying scale parameters for the main latent y . This hyperprior is still one of the standard baselines in learned image compression and is the backbone used in this project.

Later work showed that stronger entropy models often matter more than small changes in the analysis and synthesis transforms. Minnen et al. [4] combined a hierarchical prior with autoregressive context modeling and achieved clear gains over the scale hyperprior. Cheng et al. [5] further improved the prior and also used attention-style components together with a discretized Gaussian mixture likelihood. These models are relevant here because they help show how much improvement can come from a stronger prior compared with a lightweight transform-side change.

My approach is deliberately simpler than Cheng et al. [5]. I do not change the entropy model, the latent dimensionality, or the likelihood family. Instead, I insert Squeeze-and-Excitation channel attention blocks [3] into the transform network while keeping the hyperprior unchanged. This makes the comparison cleaner, but it also limits how much improvement I should expect. In the end, my results follow the literature: stronger priors provide larger improvements than the small SE change tested here.

3 Data

Table 1 summarizes the datasets present in the repository and clarifies which ones were used in the final runs. The proposal planned to train on DIV2K and evaluate on Kodak. The final implementation instead uses local CLIC image folders for training and validation. This change is visible both in the YAML configuration and the dataset loader used by `train.py`. The repository still contains DIV2K assets and a download helper, but they are not part of the final reported pipeline.

The preprocessing is simple and consistent with patch-based compression training. For training, each image is converted to RGB, reflect-padded if it is smaller than the patch size, randomly cropped to 256×256 , and randomly flipped horizontally. Validation uses the same patch size but replaces random cropping with a center crop. No per-channel normalization is applied beyond converting the image to a $[0, 1]$ tensor. Kodak evaluation uses full-resolution PNG images; each image is reflect-padded to a multiple of 64 for the



Figure 1: Patch-based preprocessing used during training. The final pipeline samples 256×256 crops from CLIC images and applies horizontal flips; validation keeps the same patch size but uses deterministic center crops.

convolutional transforms, and metrics are computed only on the original unpadded region.

This pipeline is reasonable for a course project, but it has two limitations. First, there is no separate held-out CLIC test set in the final setup; Kodak is the only external benchmark. Second, because the model is trained with MSE loss but also reported with MS-SSIM, the MS-SSIM numbers should be seen as a secondary evaluation rather than a directly optimized target.

4 Methods

4.1 Baseline Model and SE Variants

The baseline is the pretrained `bmsbj2018_hyperprior` model from CompressAI [6], which corresponds to the scale hyperprior of Ballé et al. [2]. For an input image x , the model computes

$$y = g_a(x), \quad (1)$$

$$z = h_a(y), \quad (2)$$

$$\hat{z} \sim p_{\hat{z}}, \quad (3)$$

$$\sigma = h_s(\hat{z}), \quad (4)$$

$$\hat{y} \sim p_{\hat{y}|\hat{z}}(\cdot | \sigma), \quad (5)$$

$$\hat{x} = g_s(\hat{y}), \quad (6)$$

where g_a and g_s are the analysis and synthesis transforms, h_a and h_s form the hyperprior branch, and the latent likelihoods define the bitrate term used during training.

My modification leaves h_a , h_s , the entropy bottleneck, and the Gaussian conditional unchanged. The only change is the insertion of Squeeze-and-Excitation (SE) blocks into the transform network. Given a feature

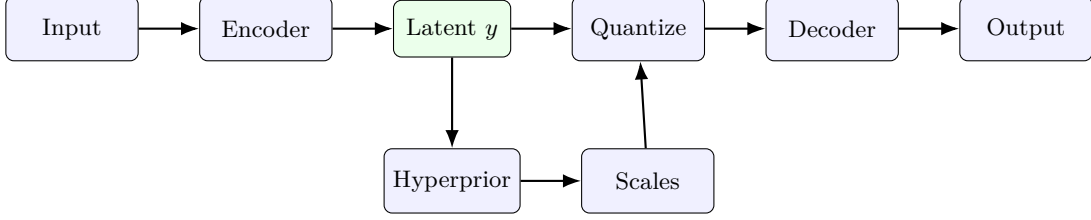


Figure 2: High-level compression system. The transform path produces the main latents y , while a separate hyperprior path predicts the scale information used by the entropy model.

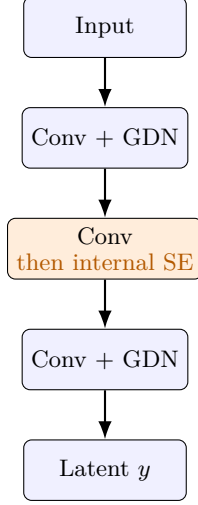


Figure 3: Encoder-side view. In the attention variants, the highlighted middle transform stage applies the SE layer internally after the second convolution.

map $f \in \mathbb{R}^{C \times H \times W}$, the SE block computes

$$s = \sigma(W_2 \text{ReLU}(W_1 \text{GAP}(f))), \quad (7)$$

$$f'_{c,h,w} = s_c \cdot f_{c,h,w}, \quad (8)$$

where GAP is global average pooling and $s \in [0, 1]^C$ rescales channels. Variant *B* inserts one SE block in the encoder after the second convolution, while variant *C* inserts the same encoder block and one additional SE block in the decoder after the second transposed convolution.

Figures 2–6 summarize the implemented model at increasing detail. The modification remains lightweight: at quality level 3, variant *B* adds only 2,184 parameters and variant *C* adds 4,368 parameters on top of the 5.08M-parameter baseline.

4.2 Initialization and Optimization

All project variants are initialized from the official pretrained scale hyperprior weights. For the SE variants, pretrained weights are transferred into the modified layer indices, while the new SE blocks are initialized near the identity: the second fully connected layer is initialized with zero weights and a bias of 5.0, so the initial sigmoid outputs are close to 1. This is a sensible choice because it reduces how much the pretrained model is disturbed when the new modules are inserted.

Training uses the standard rate-distortion objective

$$\mathcal{L} = \lambda D(x, \hat{x}) + R(\hat{y}) + R(\hat{z}), \quad (9)$$

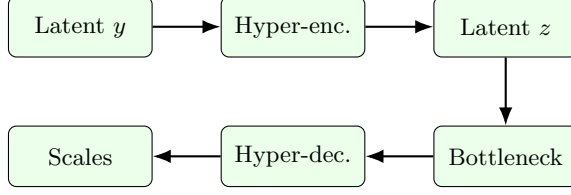


Figure 4: Entropy side-information path. This branch is unchanged across A , A_{ft} , B , and C .

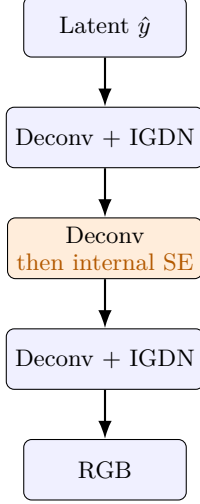


Figure 5: Decoder-side view. Only variant C modifies the highlighted middle decoder stage, where the SE layer is applied internally after the second deconvolution.

where D is MSE in the main experiments and

$$R = -\frac{1}{HW} \sum \log_2 p(\hat{y}) - \frac{1}{HW} \sum \log_2 p(\hat{z}). \quad (10)$$

The implementation follows the standard CompressAI convention and scales the MSE distortion by 255^2 . Four operating points are trained with $\lambda \in \{0.0018, 0.0035, 0.0067, 0.013\}$, matching quality levels 1–4.

Optimization uses Adam with separate learning rates for backbone and SE parameters: 10^{-4} for pretrained backbone weights, 10^{-3} for SE parameters, and 10^{-3} for the auxiliary entropy-model quantiles. The scheduler is `ReduceLROnPlateau` with factor 0.1 and patience 10. Runs stop after 100 epochs at most, with early stopping after 20 epochs without validation-loss improvement. Batch size is 32, patch size is 256×256 , and training is deterministic with seed 42.

4.3 Freeze-Mode Ablations and Evaluation Protocol

Beyond the proposal, the final repository includes three training modes: `none`, `frozen_hyperprior`, and `attention_only`. The first updates all trainable weights, the second freezes the hyperprior path, and the third updates only the SE parameters. These ablations help answer whether the SE modules are useful as small adapters or only when the full model is allowed to adapt.

For evaluation, the repository computes both estimated bitrate from latent likelihoods and true coded bitrate from actual entropy coding. The final report uses true coded bitrate for all main conclusions. This matters because the differences among A_{ft} , B , and C are very small: for the project variants, coded bitrate exceeds estimated bitrate by only 1.56×10^{-4} to 1.10×10^{-3} bpp, but that is already similar in size to the observed gaps between the curves.

The final evaluation code also measures runtime. A forward pass is timed on the main CUDA model with

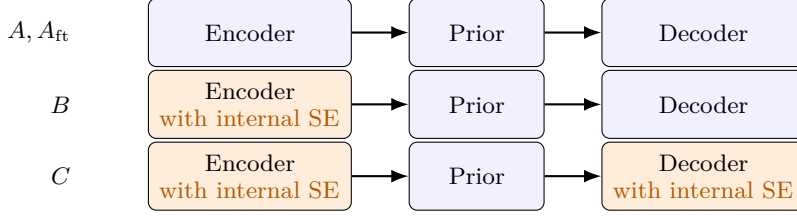


Figure 6: Variant comparison. The project modifies only the transform network: B embeds SE inside the encoder pipeline, and C embeds SE inside both encoder and decoder pipelines.

Table 2: Implemented model variants used in the report. Parameter counts come from the final evaluation summaries.

Variant	Description	Parameters
A	Official pretrained scale hyperprior reference	5,075,843
A_{ft}	Fine-tuned vanilla hyperprior control baseline	5,075,843
B	Encoder-only SE attention	5,078,027
C	Encoder+decoder SE attention	5,080,211

synchronization before and after `model(x)`. True encoding and decoding times are measured separately using `compress()` and `decompress()` on a CPU copy of the model, because the entropy-coding path in CompressAI is CPU-based in this setup. I report three timing numbers: forward time, encode time, and decode time, along with the codec total (encode+decode). These measurements are useful for checking the “lightweight” claim, but they should still be interpreted as machine-specific rather than universal throughput numbers.

5 Experiments

5.1 Experimental Setup

The main project comparison uses the four Kodak operating points corresponding to qualities 1–4. I report PSNR, MS-SSIM, and coded bits per pixel. The fair comparison point is the fine-tuned vanilla baseline A_{ft} , not the frozen official pretrained model A , because B and C are themselves fine-tuned from pretrained weights.

The repository also evaluates four stronger pretrained reference models from the CompressAI zoo: the factorized prior baseline, MBT 2018 mean, MBT 2018, and Cheng 2020 anchor. These are not direct baselines for my main hypothesis, but they help show how large an improvement really is. If a small SE block only changes BD-rate by a few hundredths of a percent while stronger priors improve it by 8–25%, that clearly affects how the result should be interpreted.

One important repository finding is that older intermediate results should not be used for the final report. The current `results_discussion.md` explicitly notes that earlier plots were misleading because they mixed stale summaries and uncoded evaluation. The final report uses the corrected summaries generated on 2026-05-28 by `evaluate.py` and `plot.py`.

5.2 Implementation Environment

All reported evaluation measurements were collected on one workstation with the following hardware:

- GPU: NVIDIA RTX 5070 Ti
- CPU: AMD Ryzen 9 9900X
- RAM: 32 GB

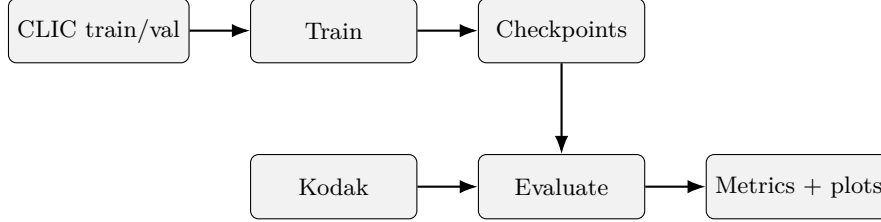


Figure 7: Training and evaluation workflow used for the final report. Runtime measurements come only from the Kodak evaluation stage.

Table 3: BD-rate relative to the fine-tuned vanilla baseline A_{ft} . Negative is better.

Model	Params (M)	PSNR BD-rate (%)	MS-SSIM BD-rate (%)
Pretrained baseline A	5.076	-0.110	-1.620
Encoder SE B	5.078	0.080	-0.150
Encoder+decoder SE C	5.080	0.170	-0.050
BMSHJ 2018 factorized	2.998	22.230	1.450
MBT 2018 mean	7.028	-8.020	-7.530
MBT 2018	14.130	-19.420	-15.560
Cheng 2020 anchor	11.833 / 26.599*	-24.870	-20.610

* In the local reference implementation, Cheng 2020 anchor uses 11.833M parameters for qualities 1–3 and 26.599M parameters for quality 4.

The runtime analysis in this report is strictly an evaluation/inference study. I report GPU forward-pass time together with CPU `compress()`/`decompress()` timing, because that is how the final evaluation code measures the practical inference path. I do not report total training time, epoch timing, or GPU-utilization summaries in the final comparison, even though the repository contains other training diagnostics, because those measurements were not part of the cleaned runtime study.

The runtime comparisons are also limited to implementations that were evaluated locally under the same script: A , A_{ft} , B , C , and the four CompressAI reference models. The repository does not include paper-reported runtimes on matching hardware, and mixing heterogeneous runtime numbers from different machines would not be methodologically clean. For that reason, all runtime discussion below is a same-machine comparison against local reference implementations rather than a claim about universal deployment speed.

5.3 Main Rate-Distortion Results

Figures 8 and 9 show the project-level rate-distortion curves. In both PSNR and MS-SSIM, the curves for A_{ft} , B , and C almost overlap. Across all four operating points, the maximum spread among these three models is below 0.009 dB and below 5.8×10^{-4} bpp. This is too small to support a strong claim that the SE modification improved compression.

Table 3 quantifies this overlap through BD-rate relative to A_{ft} . The encoder-only SE model B is slightly worse in PSNR BD-rate and marginally better in MS-SSIM BD-rate; the encoder+decoder model C is slightly worse in both metrics. In practice, these numbers are so small that they are better described as a tie than as a meaningful gain. By contrast, MBT 2018 and Cheng 2020 provide clearly better RD behavior, which is consistent with the literature on stronger context and prior modeling.

The presence of A in Table 3 is also informative. It is slightly better than A_{ft} , which suggests that local fine-tuning on the adopted CLIC setup did not improve the official pretrained model very much. This makes the negative result on SE attention easier to understand: the baseline itself is already strong, and the transform-side modification is small.

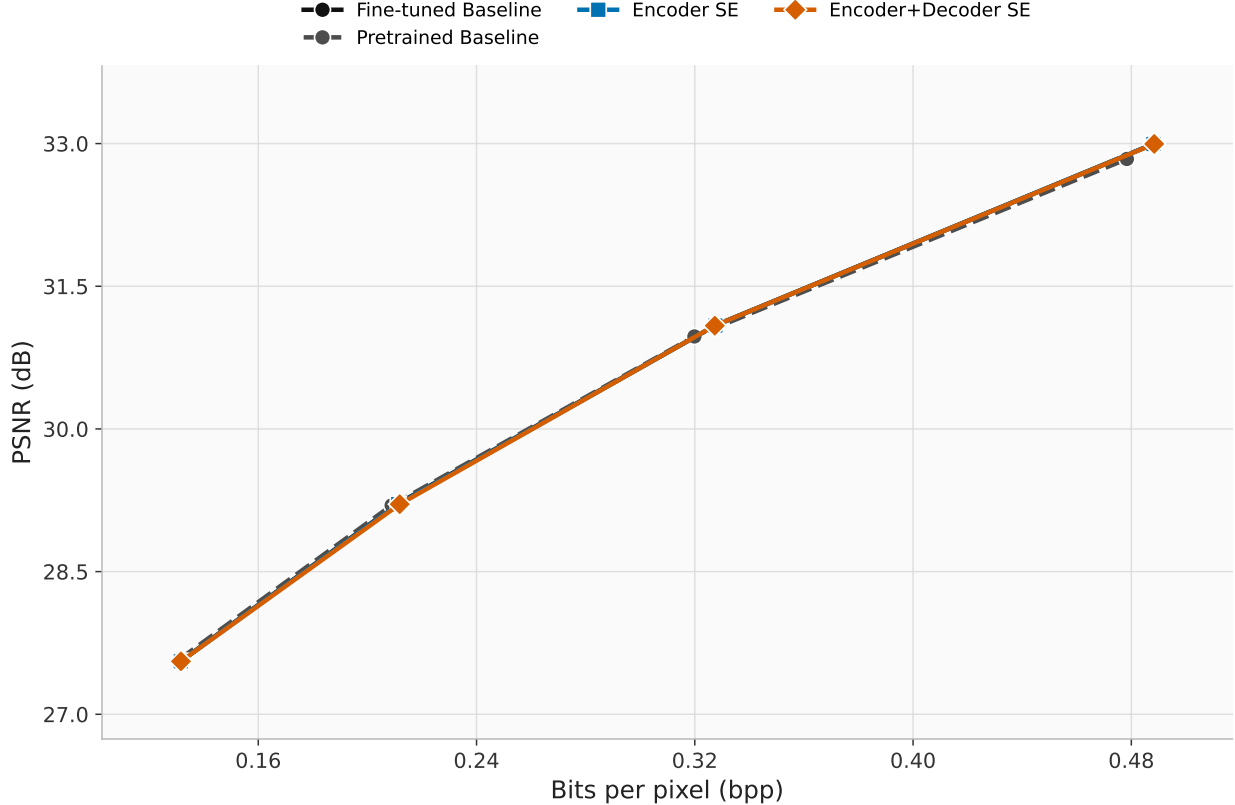


Figure 8: PSNR versus coded bitrate for the main project variants. The three curves are almost indistinguishable on Kodak.

5.4 Lightweight Parameter and Runtime Tradeoff

The SE modification was intended to be lightweight, so it is important to check not only BD-rate but also parameter count and evaluation runtime. Table 4 reports average Kodak timing across the four operating points for the main project comparison. The key observation is that the parameter overhead is indeed negligible: variant *B* adds only 0.043% parameters over A_{ft} , and variant *C* adds 0.086%. Forward-pass latency is also close, with a 1.44% increase for *B* and a 2.33% increase for *C* relative to A_{ft} .

The codec-side timings tell a slightly more detailed story. Variant *B* remains essentially tied with A_{ft} , but it is modestly slower overall (+1.57% total encode+decode time). Variant *C* is again very close and in this rerun is slightly faster than A_{ft} (−1.17% total codec time), mainly because its average decode time is a bit lower. Given how small these differences are, the safest claim is not that SE improves speed, but that the SE additions do not impose a meaningful runtime penalty in the tested setup. Combined with the BD-rate table, this supports a clear conclusion: the modification is lightweight in both parameter count and wall-clock behavior, but it still does not buy a measurable compression gain.

In practical terms, the project models all remain in roughly the same evaluation regime: about 5.9–6.1 images per second for full encode+decode on Kodak, corresponding to about 2.33–2.39 megapixels per second. This means the SE additions are genuinely lightweight from an inference perspective even though they do not improve rate-distortion quality.

The broader runtime comparison against local reference implementations is more revealing. Figure 12 shows that the factorized prior, the project variants, and MBT mean all lie in a narrow band between 155 and 170 ms per image. By contrast, the autoregressive MBT 2018 and Cheng 2020 models are much slower under the same evaluation script: 16.1× and 16.9× slower than A_{ft} , respectively. This is the main practical tradeoff visible in the repository: stronger priors do improve rate-distortion performance, but they can impose a large

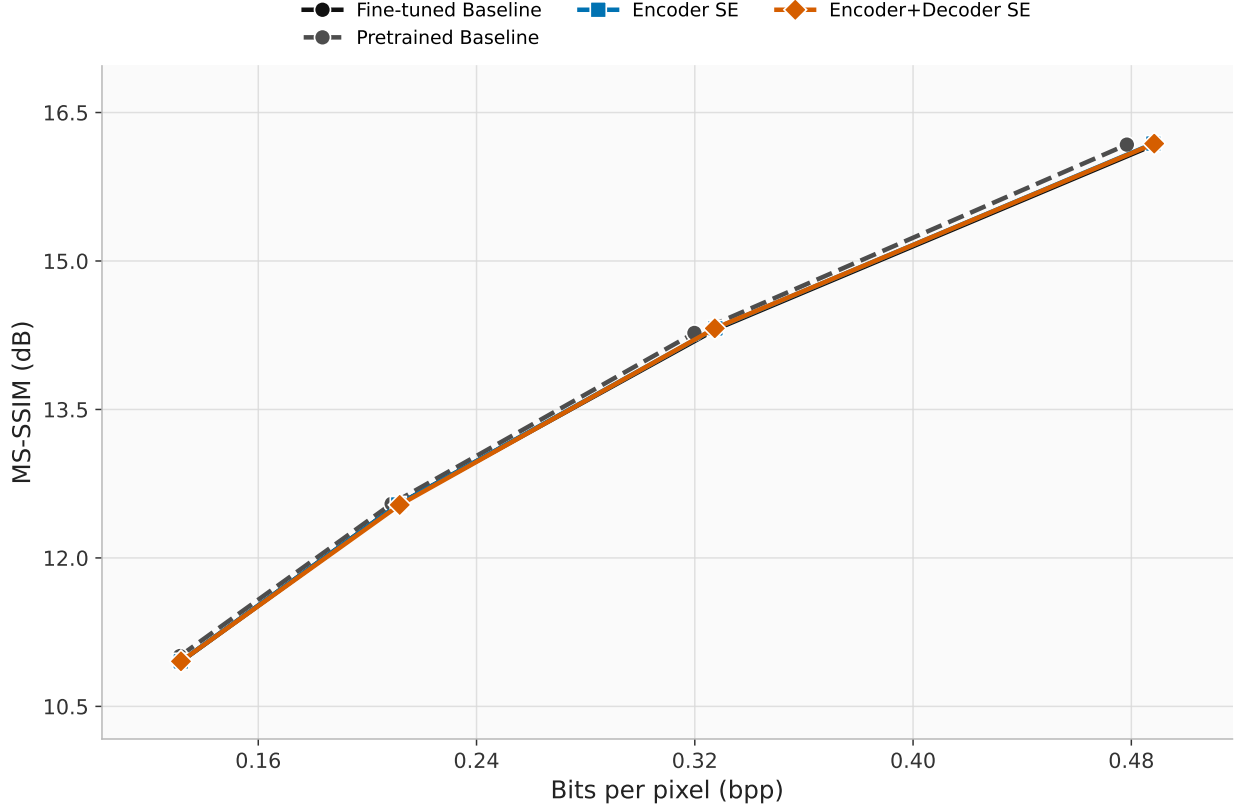


Figure 9: MS-SSIM versus coded bitrate for the same three variants. The ordering is again effectively a tie.

Table 4: Lightweight tradeoff for the main project variants, averaged over Kodak qualities 1–4. Timings come from the corrected 2026-05-28 evaluation pipeline; forward pass is timed on GPU, while `compress()`/`decompress()` are timed on CPU.

Variant	Params (M)	Param Δ (%)	Forward ms	Encode ms	Decode ms	Codec total ms	Codec Δ (%)
A_{ft}	5.076	0.000	11.390	73.340	92.850	166.180	0.000
B	5.078	0.043	11.560	74.950	93.840	168.790	1.570
C	5.080	0.086	11.660	73.050	91.200	164.250	−1.170

evaluation-time penalty.

Figure 13 summarizes the bitrate-runtime tradeoff using only same-machine measurements. The project variants cluster tightly near the fine-tuned baseline, while MBT 2018 and Cheng 2020 move toward lower average bitrate at much higher runtime cost. For Cheng 2020 anchor, this mean runtime is averaged over qualities 1–4, and the quality 4 model uses a larger local reference implementation than qualities 1–3. Paper-level runtime baselines were not used here because the repository does not provide directly comparable measurements on matching hardware.

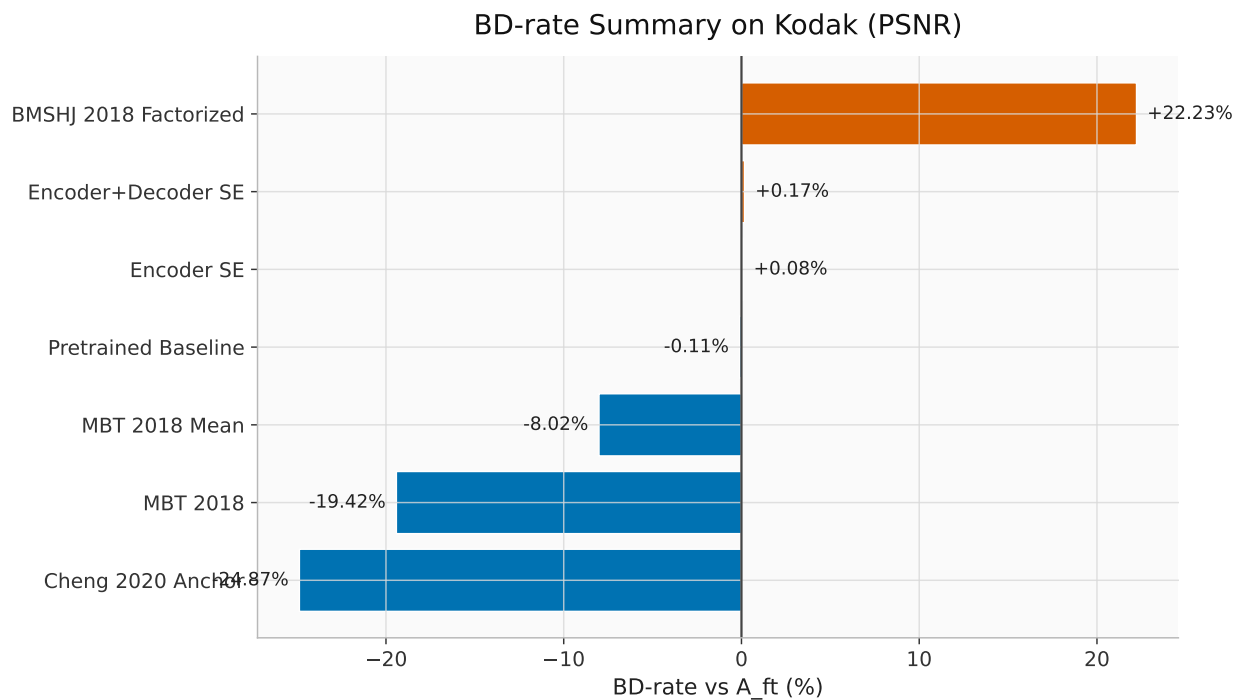


Figure 10: PSNR BD-rate against the fine-tuned vanilla baseline. This view is easier to read than the full multi-model RD curve and makes the large gap between the SE variants and stronger reference priors immediately visible.

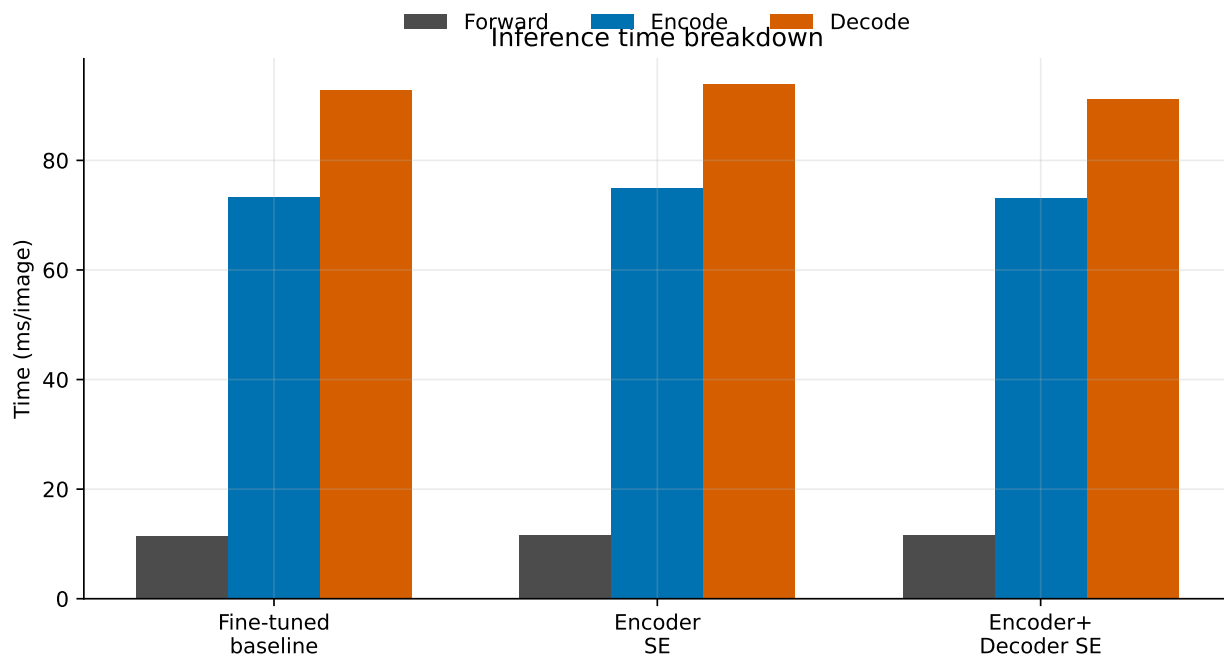


Figure 11: Evaluation-time breakdown for the three main project variants. The forward pass is small compared with entropy coding, and the SE variants stay close to the fine-tuned baseline in all three components.

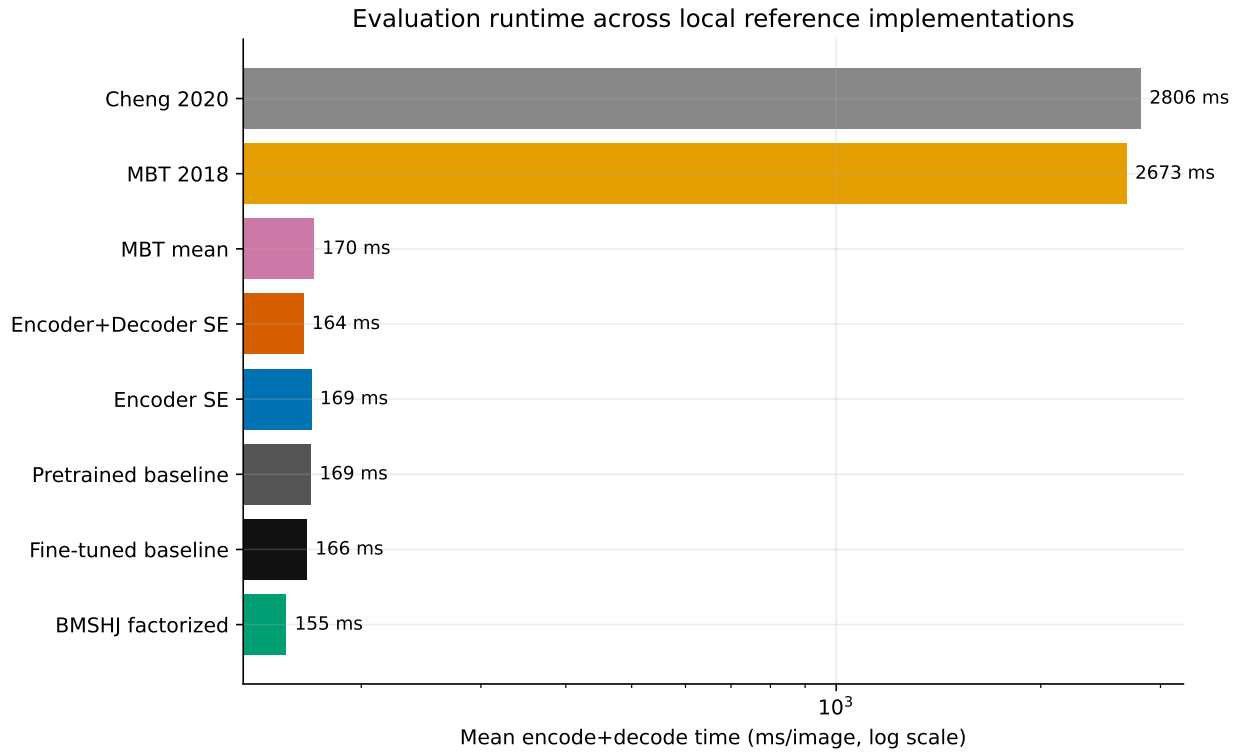


Figure 12: Mean encode+decode runtime across locally evaluated reference implementations. The log scale is necessary because the autoregressive/context-heavy models are much slower than the scale-hyperprior family.

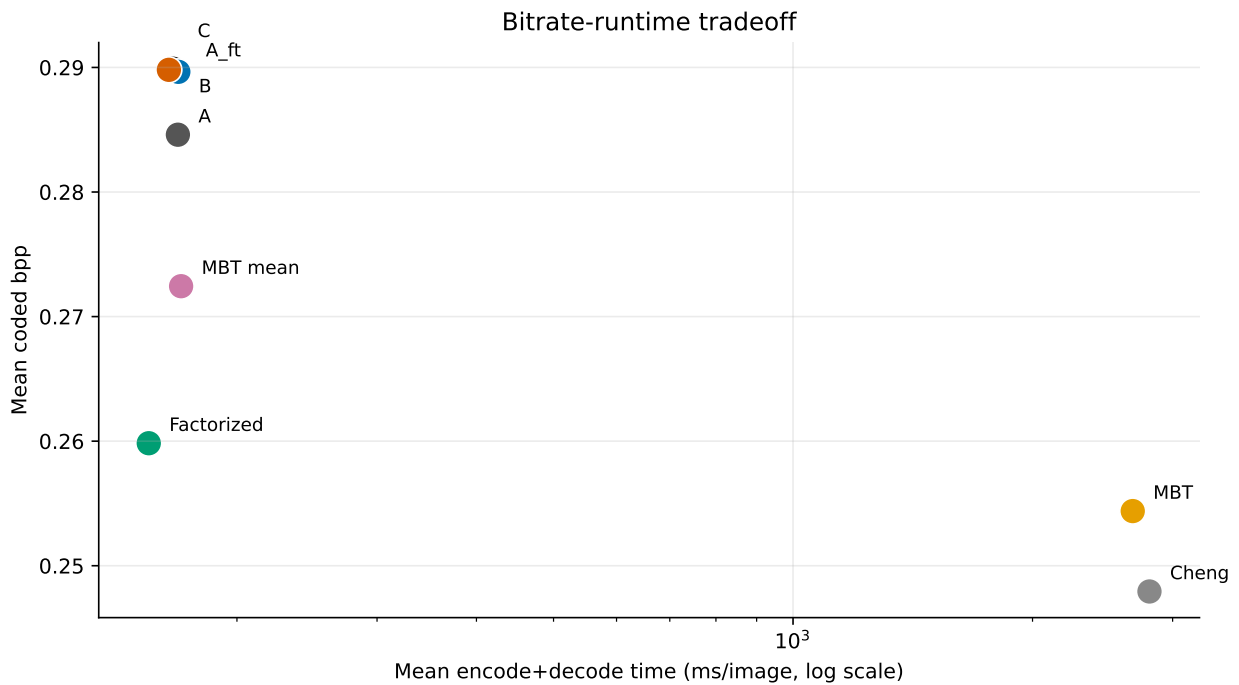


Figure 13: Bitrate-runtime tradeoff using mean Kodak measurements. The local reference implementations provide a clean practical comparison without mixing hardware-dependent runtime numbers from the literature.

5.5 Freeze-Mode Ablations

The freeze-mode ablations are scientifically useful because they test whether SE blocks act as effective adapters. Table 5 reports the mid-rate operating point ($\lambda = 0.0067$, quality 3), where differences are easiest to inspect without extreme low- or high-rate behavior.

Two patterns stand out. First, **attention_only** peaks much earlier and underperforms the fully trainable runs, especially for B . This suggests that the SE modules alone do not have enough capacity to retune the compression model in a useful way. Second, **frozen_hyperprior** is usually close to **none** but not better, so freezing the prior does not reveal a hidden benefit of the transform-side attention either.

Figure 14 shows the validation curves for the **none** runs at quality 3. The three models converge in almost the same region, which matches the final Kodak evaluation. There is no visible sign that the SE variants settle at a distinctly better optimum.

5.6 Qualitative Reconstruction Comparison

Figure 15 shows one Kodak example and a zoomed crop. All three project variants reconstruct the image well at the chosen operating point, but the differences among them are visually minor. The main artifacts are the same across models: slight smoothing around the printed text on the hat, softened fabric texture, and mild blur along edges.

This qualitative result is consistent with the quantitative one. The SE blocks do not fail catastrophically; they simply do not produce a visible gain large enough to stand out from ordinary run-to-run or operating-point differences.

5.7 What Would Improve the Evidence

Although the repository is in a reasonably good state for a term project, the experimental evidence is still limited in several ways.

- Only one random seed is reported. Since the observed differences are extremely small, multi-seed training would be necessary to decide whether the tiny BD-rate gaps are real.
- All learned variants are trained with MSE. Training a parallel MS-SSIM-optimized set would make the MS-SSIM evaluation more meaningful.
- Runtime is measured only on one local hardware/software stack. The current timing table is useful for relative comparison, but a stronger systems claim would require repeated measurements across multiple runs and cleaner CPU/GPU benchmarking conditions.
- The project changes only one SE insertion location. A broader attention-placement ablation could test whether the null result is due to weak attention or simply the chosen location.

Within the current repository, however, the central conclusion is already stable: stronger entropy models matter much more than the tested SE augmentation.

One possible reason is that my SE design is much simpler than the attention used in stronger models such as Cheng 2020. In my model, the SE block uses global average pooling and gives one weight to each channel, so the same scaling is used everywhere in the feature map. This is lightweight and easy to add, but it may be too simple for image compression, where different parts of an image can look very different. Cheng 2020 uses a stronger overall model with more flexible attention and a better prior model. Because of that, its better rate-distortion results should not be taken to mean that any attention block will help. A safer interpretation is that simple global channel reweighting was not enough in my setting, while stronger attention and stronger entropy modeling can be more useful.

Table 5: Mid-rate ablation at quality 3 ($\lambda = 0.0067$). Best epoch is read from the saved checkpoint metadata.

Variant	Freeze mode	coded bpp	PSNR (dB)	Best epoch
A_{ft}	none	0.328	31.094	86.000
A_{ft}	frozen hyperprior	0.328	31.092	84.000
B	attention only	0.320	30.976	14.000
B	none	0.328	31.087	77.000
B	frozen hyperprior	0.327	31.075	87.000
C	attention only	0.322	30.995	36.000
C	none	0.327	31.086	93.000
C	frozen hyperprior	0.328	31.080	68.000

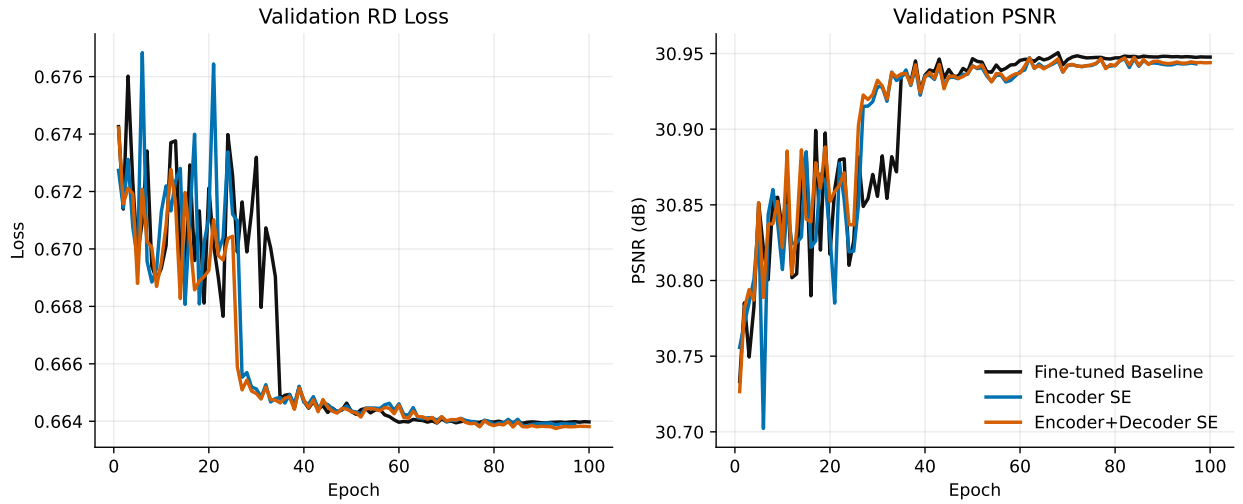


Figure 14: Validation convergence at quality 3 for the fully trainable runs. The curves track each other closely, and the final PSNR values are nearly identical.



Figure 15: Qualitative comparison on Kodak image kodim03 at quality 3. The SE variants do not recover clearly sharper structure than the fine-tuned vanilla baseline.

6 Conclusion

This project implemented the proposed idea successfully at the engineering level. I built two SE-augmented variants of the scale hyperprior, transferred pretrained weights carefully, added fair control baselines, trained multiple freeze-mode ablations, and evaluated the models with true coded bitrate on Kodak.

The final experimental outcome is negative but still informative. After correcting the evaluation pipeline and comparing against a fine-tuned vanilla control, the SE variants do not provide a meaningful rate-distortion improvement. Their Kodak curves are essentially on top of the baseline, while stronger pretrained priors such as MBT 2018 and Cheng 2020 show clearly larger gains. The main lesson is that, for learned compression, improving the prior model is usually more helpful than inserting a very small amount of channel attention into the transforms.

For a course project, this is still a useful result. The implementation work was substantial, the comparisons are reasonably clean, and the report can make an honest claim: under this training and evaluation setup, lightweight SE attention did not noticeably improve a scale-hyperprior image compressor.

References

- [1] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations*, 2017.
- [2] Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.
- [3] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7132–7141, 2018.
- [4] David Minnen, Johannes Ballé, and George Toderici. Joint autoregressive and hierarchical priors for learned image compression. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [5] Zhengxue Cheng, Heming Sun, Masaru Takeuchi, and Jiro Katto. Learned image compression with discretized Gaussian mixture likelihoods and attention modules. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7939–7948, 2020.
- [6] Jean Bégaint, Fabien Racapé, Simon Feltman, and Akshay Pushparaja. CompressAI: a PyTorch library and evaluation platform for end-to-end compression research. *arXiv preprint arXiv:2011.03029*, 2020.
- [7] Radu Timofte, Eirikur Agustsson, Luc Van Gool, Ming-Hsuan Yang, and others. NTIRE 2017 challenge on single image super-resolution: dataset and study. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 114–125, 2017.
- [8] Workshop and Challenge on Learned Image Compression (CLIC). <https://www.compression.cc/>. Accessed May 2026.
- [9] Kodak Lossless True Color Image Suite. <https://r0k.us/graphics/kodak/>. Accessed May 2026.